

E-Commerce Database Management System Project Documentation

Project Overview

This project develops a simple E-Commerce Database Management System to manage customers, products, sellers, orders, and payments using SQL.

E-Commerce Business Process

Customer registers, browses products, places orders, makes payments, and sellers manage products and deliveries.

Customer Management Requirements

Store customer details, maintain order history, manage payments, and provide unique customer identification.

Major Entities

Customer, Product, Seller, Order, Payment.

Entities and Attributes

Customer(CustomerID, Name, Email, Phone)

Product(ProductID, ProductName, Price, Stock)

Seller(SellerID, SellerName, Contact)

Orders(OrderID, CustomerID, ProductID, Quantity, OrderDate)

Payment(PaymentID, OrderID, Amount, PaymentMethod)

Relationships Between Entities

A Customer places Orders. Orders contain Products. Sellers provide Products. Payments are made for Orders.

ER Diagram

Customer -> Orders -> Product

Seller -> Product

Orders -> Payment

Functional Requirements

Add, view, update and delete customer, product, order, seller and payment records.

Review Questions & Answers

Q: What is a Primary Key? A: A unique identifier for each record.

Q: What is a Foreign Key? A: A key used to establish relationships between tables.

SQL Script

-- 1. Create Database

```
CREATE DATABASE EcommerceDB;
```

```
-- 2. Select Database
```

```
USE EcommerceDB;
```

```
-- 3. Create Customer Table
```

```
CREATE TABLE Customer (  
    CustomerID INT PRIMARY KEY,  
    Name VARCHAR(50) NOT NULL,  
    Email VARCHAR(100) UNIQUE,  
    Phone VARCHAR(15)  
);
```

```
-- 4. Create Seller Table
```

```
CREATE TABLE Seller (  
    SellerID INT PRIMARY KEY,  
    SellerName VARCHAR(50) NOT NULL,  
    Contact VARCHAR(15)  
);
```

```
-- 5. Create Product Table
```

```
CREATE TABLE Product (  
    ProductID INT PRIMARY KEY,
```

```
    ProductName VARCHAR(50) NOT NULL,  
    Price DECIMAL(10,2) NOT NULL,  
    Stock INT NOT NULL,  
    SellerID INT,  
    FOREIGN KEY (SellerID) REFERENCES Seller(SellerID)  
);
```

-- 6. Create Orders Table

```
CREATE TABLE Orders (  
    OrderID INT PRIMARY KEY,  
    CustomerID INT,  
    ProductID INT,  
    Quantity INT NOT NULL,  
    OrderDate DATE,  
    FOREIGN KEY (CustomerID) REFERENCES Customer(CustomerID),  
    FOREIGN KEY (ProductID) REFERENCES Product(ProductID)  
);
```

-- 7. Create Payment Table

```
CREATE TABLE Payment (  
    PaymentID INT PRIMARY KEY,  
    OrderID INT,  
    Amount DECIMAL(10,2) NOT NULL,  
    PaymentMethod VARCHAR(30) NOT NULL,
```

```
FOREIGN KEY (OrderID) REFERENCES Orders(OrderID)
);
```

-- 8. Insert Customer Data

```
INSERT INTO Customer (CustomerID, Name, Email, Phone) VALUES
(1, 'Arun', 'arun@gmail.com', '9876543210'),
(2, 'Priya', 'priya@gmail.com', '9876543211'),
(3, 'Rahul', 'rahul@gmail.com', '9876543212'),
(4, 'Divya', 'divya@gmail.com', '9876543213'),
(5, 'Kavin', 'kavin@gmail.com', '9876543214'),
(6, 'Anitha', 'anitha@gmail.com', '9876543215'),
(7, 'Vijay', 'vijay@gmail.com', '9876543216'),
(8, 'Meena', 'meena@gmail.com', '9876543217'),
(9, 'Ravi', 'ravi@gmail.com', '9876543218'),
(10, 'Keerthi', 'keerthi@gmail.com', '9876543219');
```

-- 9. Insert Seller Data

```
INSERT INTO Seller (SellerID, SellerName, Contact) VALUES
(1, 'Tech World', '90000000001'),
(2, 'Fashion Hub', '90000000002'),
(3, 'Home Store', '90000000003'),
(4, 'Book House', '90000000004'),
(5, 'Mobile Zone', '90000000005');
```

-- 10. Insert Product Data

INSERT INTO Product (ProductID, ProductName, Price, Stock, SellerID) VALUES

(1, 'Laptop', 55000.00, 10, 1),

(2, 'Mobile Phone', 25000.00, 15, 5),

(3, 'Headphones', 2000.00, 25, 1),

(4, 'T-Shirt', 799.00, 30, 2),

(5, 'Shoes', 1999.00, 20, 2),

(6, 'Table Lamp', 1200.00, 12, 3),

(7, 'Chair', 3500.00, 8, 3),

(8, 'Novel Book', 499.00, 40, 4),

(9, 'Keyboard', 1500.00, 18, 1),

(10, 'Smart Watch', 4500.00, 14, 5);

-- 11. Insert Orders Data

INSERT INTO Orders (OrderID, CustomerID, ProductID, Quantity, OrderDate) VALUES

(1, 1, 1, 1, '2026-09-01'),

(2, 2, 2, 1, '2026-09-02'),

(3, 3, 3, 2, '2026-09-02'),

(4, 4, 4, 3, '2026-09-03'),

(5, 5, 5, 1, '2026-09-03'),

(6, 6, 6, 2, '2026-09-04'),

(7, 7, 7, 1, '2026-09-05'),

(8, 8, 8, 2, '2026-09-05'),

(9, 9, 9, 1, '2026-09-06'),

```
(10, 10, 10, 1, '2026-09-07');
```

```
-- 12. Insert Payment Data
```

```
INSERT INTO Payment (PaymentID, OrderID, Amount, PaymentMethod) VALUES
```

```
(1, 1, 55000.00, 'UPI'),
```

```
(2, 2, 25000.00, 'Card'),
```

```
(3, 3, 4000.00, 'UPI'),
```

```
(4, 4, 2397.00, 'Cash'),
```

```
(5, 5, 1999.00, 'Card'),
```

```
(6, 6, 2400.00, 'UPI'),
```

```
(7, 7, 3500.00, 'Cash'),
```

```
(8, 8, 998.00, 'Card'),
```

```
(9, 9, 1500.00, 'UPI'),
```

```
(10, 10, 4500.00, 'Card');
```

```
-- 13. SELECT Queries
```

```
SELECT * FROM Customer;
```

```
SELECT * FROM Seller;
```

```
SELECT * FROM Product;
```

```
SELECT * FROM Orders;
```

```
SELECT * FROM Payment;
```

```
-- 14. UPDATE Query
```

```
UPDATE Customer
```

```
SET Phone = '9999999999'
```

```
WHERE CustomerID = 1;
```

```
-- Check Updated Data
```

```
SELECT * FROM Customer
```

```
WHERE CustomerID = 1;
```

```
-- 15. DELETE Query
```

```
DELETE FROM Customer
```

```
WHERE CustomerID = 10;
```

```
-- Check Deleted Data
```

```
SELECT * FROM Customer;
```

```
-- 16. Create View
```

```
CREATE VIEW CustomerView AS
```

```
SELECT CustomerID, Name, Email, Phone
```

```
FROM Customer;
```

```
-- 17. Display View
```

```
SELECT * FROM CustomerView;
```